

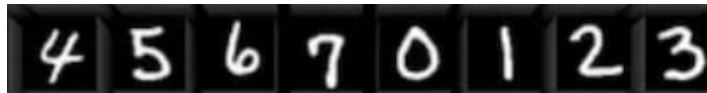
# R4.A.L1 – Introduction à l'intelligence artificielle

## Utilisation d'un perceptron multicouche (MLP) pour la reconnaissance de caractères manuscrits

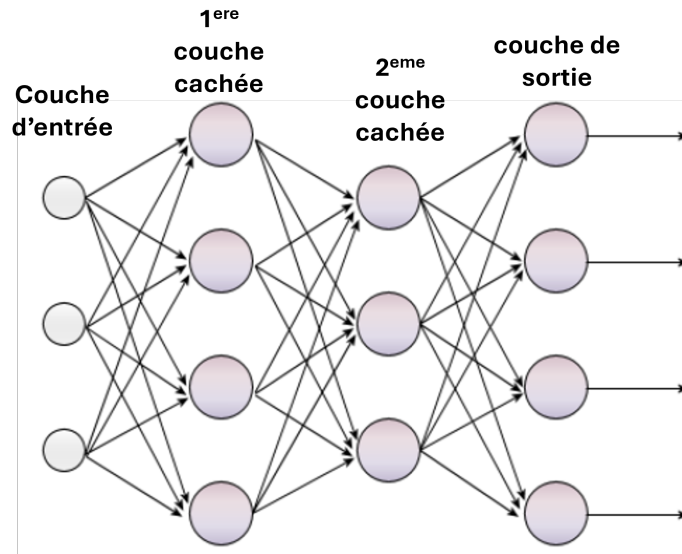
Nous allons entraîner un modèle capable de reconnaître les chiffres manuscrits. Pour cela, nous utiliserons la base de données MNIST, qui contient un ensemble d'images de caractères manuscrits. Nous utiliserons un réseau de neurones, plus précisément un perceptron multicouche (MLP), pour classer les images de chiffres dans l'une des 10 catégories.

### 1 Chargement des données

Nous devons d'abord récupérer et traiter les données d'entraînement. Pour cela, nous allons utiliser MNIST, un jeu de données annoté composé d'images de chiffres manuscrits de 0 à 9 (ainsi que leur label correspondant entre 0 et 9).



1. Télécharger les données depuis [https://git-disl.github.io/GTDLBench/datasets/mnist\\_datasets/](https://git-disl.github.io/GTDLBench/datasets/mnist_datasets/), sous l'onglet "Download\_MNIST\_CSV", qui fournit les données sous forme de fichiers CSV. Ces fichiers contiennent des images en noir et blanc de taille 28x28 pixels, séparées en un ensemble d'entraînement et un ensemble de test.
2. Ouvrir un fichier CSV avec un éditeur de texte et observer son contenu. Comment sont représentées les données (où sont l'image et le label) ? Quelle est la plage des valeurs des pixels (valeur minimale et maximale) ?
3. Charger le fichier CSV des données d'entraînement en Python en utilisant :
  - La fonction NumPy `np.loadtxt(chemin, delimiter=',', dtype=int)`. Il est nécessaire d'indiquer `delimiter=','` pour éviter que les nombres soient mal interprétés, ainsi que `dtype=int` pour forcer un tableau d'entiers.
  - Ou la fonction Pandas `pandas.read_csv(chemin, header=None)`, qui retourne un DataFrame. L'argument `header=None` empêche l'interprétation de la première ligne comme en-tête.
4. Séparer les labels des images en créant deux tableaux : `y_train` pour les labels (entiers entre 0 et 9) et `X_train` pour les données d'image. Normaliser les valeurs des pixels entre 0 et 1. Combien de données d'entraînement avons-nous ? Quelle est la dimension des images ?
5. Afficher quelques images d'entraînement avec la fonction `plt.imshow()` de Matplotlib. Vérifier que la `shape` des données correspond bien à une image 28x28 pixels en utilisant `numpy.reshape()`.



**Perceptron multicouche à 3 entrées et 4 sorties, et 2 couches cachées**

## 2 Entraînement du modèle

Le modèle utilisé pour classifier les chiffres manuscrits sera un réseau de neurones, plus précisément un perceptron multicouche (MLP). Nous utiliserons l'implémentation de **sklearn**.

1. Créer un modèle **MLPClassifier** de **sklearn.neural\_network**. Ce modèle gère déjà la fonction de perte et le nombre de neurones en entrée et sortie. Les hyperparamètres à configurer sont :
  - **hidden\_layer\_sizes** : nombre de couches cachées et nombre de neurones par couche.
  - **activation** : fonction d'activation (par défaut, ReLU).
  - **learning\_rate\_init** : taux d'apprentissage pour la descente de gradient.
  - **max\_iter** : nombre d'epochs d'apprentissage. Ce nombre correspond au nombre de fois que la base de données d'entraînement sera parcourue intégralement pendant l'apprentissage par descente de gradient. Choisir une valeur faible (ex. 5) car l'entraînement itératif par descente de gradient peut être long.
2. Entraîner le modèle avec la méthode **fit()**.
3. Après l'entraînement, afficher la courbe de l'erreur d'apprentissage grâce à l'attribut **loss\_curve\_** du modèle, et la fonction **plot()** de Matplotlib. L'erreur diminue-t-elle bien au fil des epochs ? Ajuster les hyperparamètres si nécessaire.

## 3 Test du modèle

Nous allons maintenant tester notre modèle sur des images qu'il n'a pas rencontrées durant l'entraînement.

1. Charger les données de test, en appliquant les mêmes pré-traitements que pour les données d'entraînement.
2. Prédire les catégories des images de test avec la méthode **predict()** du modèle.

3. Afficher quelques images de test avec leur label prédit, en utilisant `imshow()` et `title()` de Matplotlib.
4. Identifier des images mal classées et les afficher de la même manière.
5. Afficher la matrice de confusion avec `confusion_matrix` de `sklearn.metrics` et `heatmap` de `seaborn` :

```
import seaborn as sns
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(test_labels, predictions)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predictions")
plt.ylabel("Vraies_valeurs")
plt.title("Matrice_de_confusion")
plt.show()
```

6. Calculer l'accuracy du modèle avec `accuracy_score` de `sklearn.metrics`.
7. Modifier les hyperparamètres du modèle et recommencer l'entraînement pour améliorer la performance.